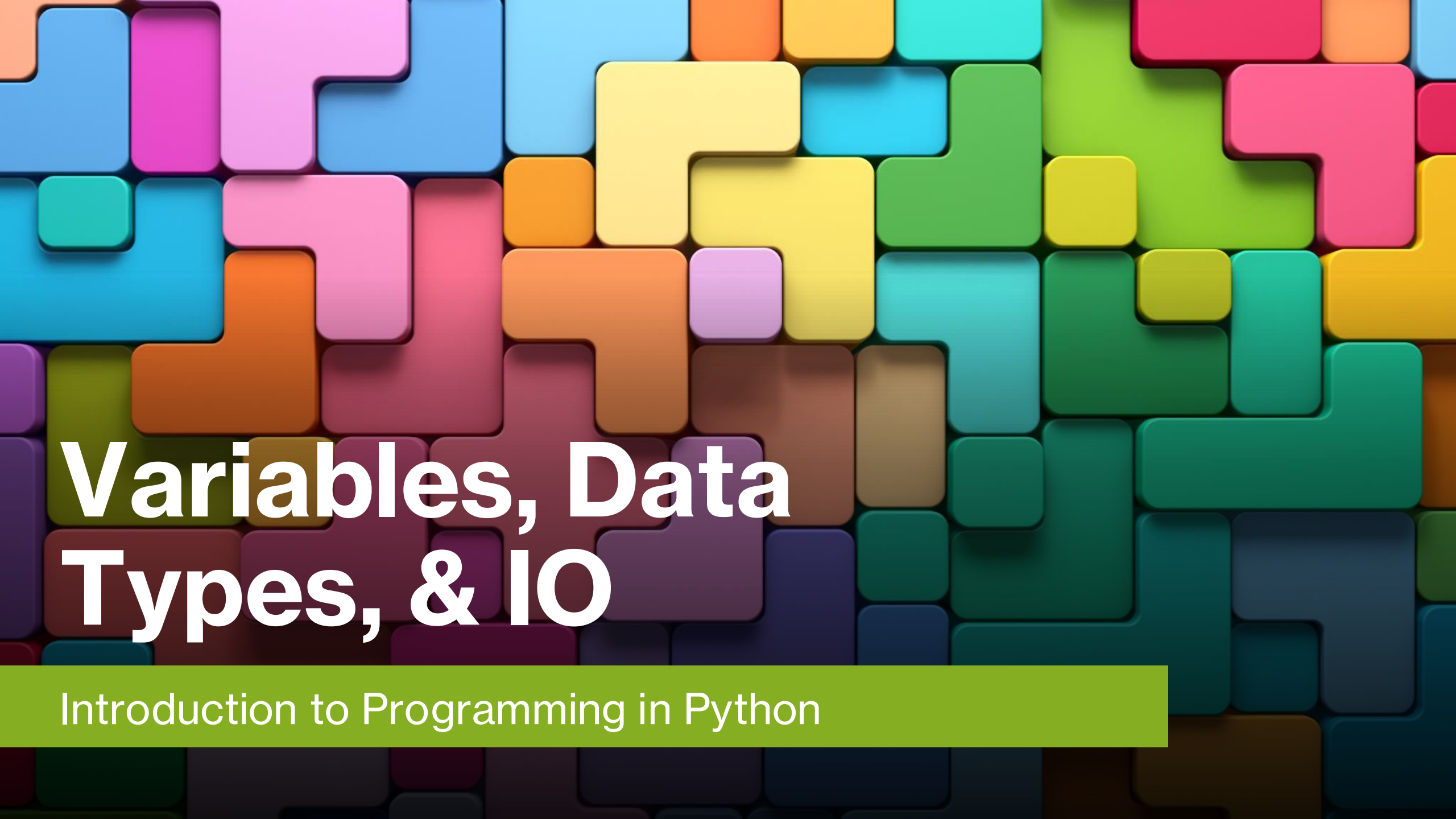




Variables, Data Types, & IO

Introduction to Programming in Python



Topics

- Designing a Program
- Comments
- Variables
- Data Types
- Output
- Input

Designing a Program

Programs must be designed before they are written

Program development cycle:

- Design the program
- Write the code
- Correct syntax errors
- Test the program
- Correct logic errors

Designing a Program

Design is the most important part of the program development cycle

Understand the program task

- Get a sense what the program is supposed to do
- Ask questions about program details
- List the requirements

Designing a Program

Determine the steps that must be taken to perform the task

- Break down required task into a series of steps
- Create an algorithm, listing logical steps that must be taken

Algorithm: set of well-defined logical steps that must be taken to perform a task

Comments

Comments are hints or notes that can be added to our code. In Python, comments start with a `#` or are wrapped between `''' '''`. Comments are completely ignored when your code is executed. It's recommended to write comments before writing code.

Example:

```
# This is a comment
```

```
''' This is a comment '''
```



Variables & Data Types

Variables

Variables are containers used to hold data in a program. Each variable is identified by its **identifier** (their name).

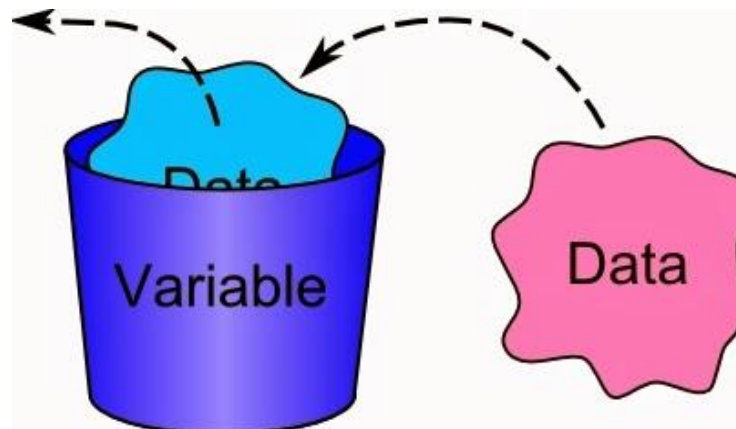
In Python, to declare (create) a variable a data value known as a **literal** must be **assigned** to it.

Literal: is a fixed data value like text or numbers.

Assigned: when a literal is stored into a variable using the **=** operator.

Example:

```
name = "John Doe"  
age = 22  
nameCopy = name
```



Variables

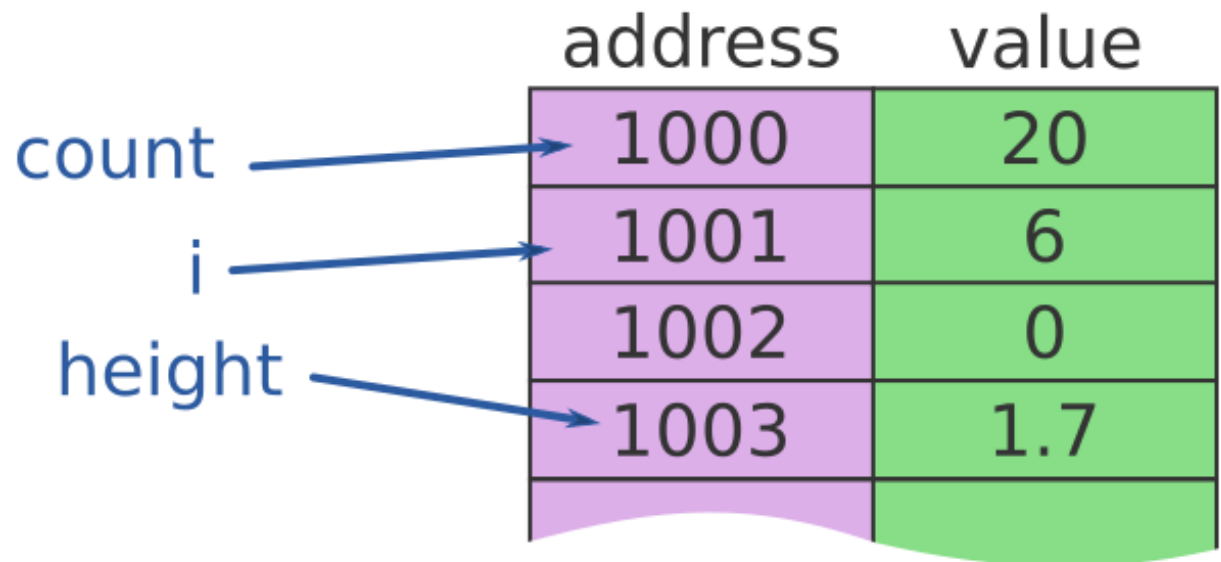
Your program uses the **identifiers** (variable name) to identify a variable, however during execution the computer uses a **memory address** that is linked to every identifier to identify a variable.

Example

count = 20

i = 6

height = 1.7



	address	value
count	1000	20
i	1001	6
	1002	0
height	1003	1.7

Variables

When declaring a variable its identifier must follow the rules below.

Variable Identifier Rules:

- Can't start with a number: **3num** ✗ **num3** ✓
- Can't have special characters, only '_': **val%** ✗ **my_val** ✓
- Can't have spaces: **full name** ✗ **fullName** ✓
- Avoid starting with a capital letter: **Age** ✗ **age** ✓
- Should be related to its use: **n = "Jon"** ✗ **name = "Jon"** ✓

Variables

The literal assigned to a variable is not permanent. A variable can always be re-assigned, and the new literal doesn't have to be the same as the previous.

Example:

```
tempVar = "Hello"
```

```
tempVar = 33
```

```
tempVar = True
```

Data Types

Data types specify the type of data that is stored inside a variable and how the program will interact with that data.

A variable's data type is assigned when a literal value is assigned to it. The literal value determines a variable's data type.

In Python, data types are known as **classes**.



Data Types

Python has built-in data types for working with data of numerical, string (text), Boolean (True/False), and sequence types.

The following slide shows all the data types Python can work with and their designated class name.

Data Types

Data Types	Classes	Description
Numeric	int, float, complex	holds numeric values
String	str	holds sequence of characters
Sequence	list, tuple, range	holds collection of items
Mapping	dict	holds data in key-value pair form
Boolean	bool	holds either True or False
Set	set, frozenset	hold collection of unique items

Data Types

The most common data type classes you will work with are **int**, **float**, **str**, and **bool**.

- **int** – holds whole numbers, **ex:** {...-2, -1, 0, 1, 2, ...}.
- **float** – holds real number, **ex:** {..., -1.99, -1.013, 0, $\sqrt{2}$, $\frac{3}{2}$, ...}.
- **str** – holds string of characters (text), **ex:** “a”, “aabb”, “Bob”.
- **bool** – holds Booleans, **ex:** True, False.

Data Types

It's important to note certain data types don't interact well with each other. For example, numerical data types cannot be summed up with string data types. We will discuss how to solve this in the next PowerPoint.

Example

`var1 = "2"`

`var2 = 2`

`var3 = var1 + var2` -> **Error**, since `var3 = "2" + 2` and we cannot add different data types



Output & Input

Output

Python uses the `print()` function to output data to the user's screen.

- **Function:** a piece of pre-written code that performs a certain function.

The data to be outputted must be given to the function as an **argument**. An argument is any data placed inside the function's parenthesis.

Example:

```
print( "hello" )  
print( 77 )  
name = "Bob"  
print( name )
```

In the example above, **"hello"**, **77**, and **name** are arguments.

Output

As seen in the previous example, the `print()` function can output data of multiple types. To output data of multiple types in the same `print()` function, you can separate the arguments using commas. There is no limit to the number of arguments a `print()` function can accept.

- **Example**

```
print("The year is", 2025) → The year is 2025
```

```
var1 = "Hello"
```

```
var2 = 23
```

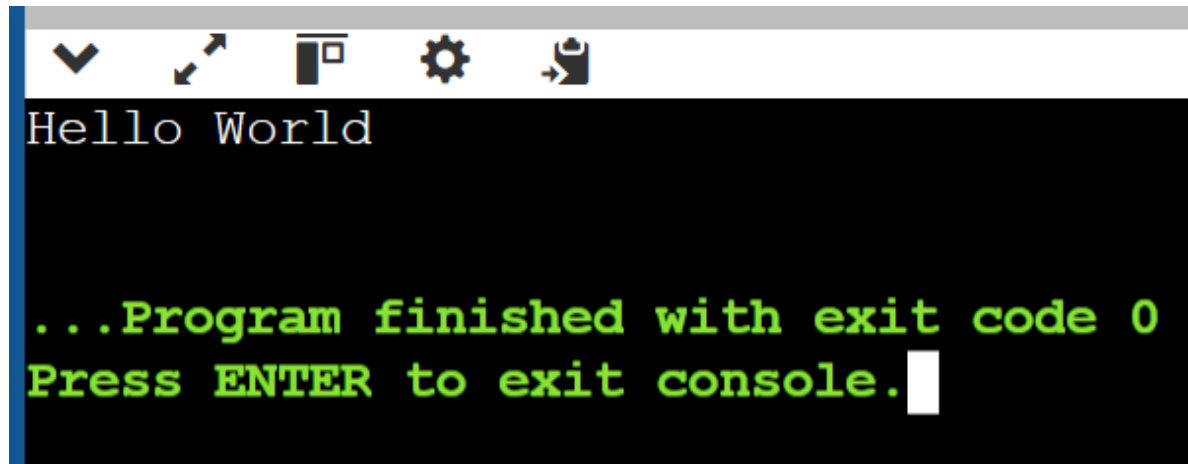
```
var3 = False
```

```
print(var1, var2, var3) → Hello 23 False
```

Output

Output data will appear to the user in their terminal or command prompt.

Terminal: a text-based interface used to enter and execute commands directly to the operating system.



```
✓ ↗ 📄 ⚙️ 📋  
Hello World  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Input

The **input()** function is used to collect data from the user. When the input() function is **called** (invoked) the program will pause and await the user's input from the terminal.

The user must hit the enter key after typing their data to submit their input.

By default, all data input by the user will be formatted as a string type.

Input

The `input()` function can take a single argument to use as **prompt**. A prompt is a message that tells the user the expected input. A prompt is not necessary for the `input()` function to work but is recommended, remember the user won't always be you.

Example

```
name = input() ← John Doe
```

```
print(name) → John Doe
```

```
name = input("What is your name? ") ← Jane Doe
```

```
print(name) → Jane Doe
```